EXPERIMENT NO: 11 **Date:** _____

EXCEPTION HANDLING

AIM

Write a program to handle exception in the case of voters age validity If age ¡ 0 raise an exception

OBJECTIVES

- To raise custom exceptions (ValueError) based on business logic.
- To wrap error prone statements inside try-except blocks.
- To intercept specific exception classes (ValueError, TypeError).

ALGORITHM

Step 1 :- Begin voter eligibility checking program with custom exception handling for negative age values.

Step 2 :- Declare function check voter age accepting numerical integer age parameter for validation evaluation.

Step 3 :- Raise ValueError exception with error message if input age is strictly less than zero.

Step 4 :- Print not eligible if age is under eighteen, or eligible if age is eighteen or above.

Step 5 :- Prompt user for integer age input inside try block and call check voter age function.

Step 6 :- Catch raised ValueError exception and print descriptive error message to terminal console.

Step 7 :- Catch TypeError exception if user provides invalid non integer input for age variable.

Step 8 :- Conclude exception handling program safely without crashing application runtime environment on invalid user inputs.

PROGRAM

```
def check_voter_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative.")
    elif age < 18:
        print("You are not eligible to vote.")
    else:
        print("You are eligible to vote.")

try:
    age = int(input("Enter your age: "))
    check_voter_age(age)
except ValueError as e:
    print(f"Error: {e}")
except TypeError:
    print("Please enter a valid integer for age.")
```

SAMPLE OUTPUT

Terminal Output

```
Enter your age:  -5
Error:  Age cannot be negative.
```

RESULT

The voter age exception handling program was executed and exception handling verified.

EXPERIMENT NO: 12 **Date:** _____

STUDENT CLASS

AIM

Write a program to create a student class and create following methods: setMarks() - to update marks, getMarks() - to get individual marks, avgMarks() - to find average of marks, showResults() - to show results of each student

OBJECTIVES

- To understand Object-Oriented Programming (OOP) concepts in Python.
- To initialize instance attributes using constructors ($_{init}$).
- To define instance methods for data manipulation, query, and formatted summary display.

ALGORITHM

Step 1 :- Initialize student management application using object oriented programming concepts to model student scores.

Step 2 :- Define Student class with constructor initializing name attribute and empty marks dictionary.

Step 3 :- Define setMarks method taking subject and mark parameters to update student marks dictionary.

Step 4 :- Define getMarks method to return individual subject score or default missing message string.

Step 5 :- Define avgMarks method calculating average by dividing sum of marks by total subjects.

Step 6 :- Define showResults method printing subject scores breakdown and computed average mark summary.

Step 7 :- Create Student object student1, set marks for subjects, and call class methods to display results.

Step 8 :- Conclude execution of object oriented student class program and destroy object instance from memory.

PROGRAM

```
class Student:
    def __init__(self, name):
        self.name = name
        self.marks = {}

    def setMarks(self, subject, mark):
        self.marks[subject] = mark

    def getMarks(self, subject):
        return self.marks.get(subject, "No marks available for this subject
    .")
```

```
11
12      def avgMarks(self):
13          if not self.marks:
14              return "No marks available."
15          total_marks = sum(self.marks.values())
16          average_marks = total_marks / len(self.marks)
17          return average_marks
18
19      def showResults(self):
20          if not self.marks:
21              print(f"No marks available for {self.name}.")
22              return
23          print(f"Results for {self.name}:")
24          for subject, mark in self.marks.items():
25              print(f"{subject}: {mark}")
26          average_marks = self.avgMarks()
27          print(f"Average Marks: {average_marks}")
28
29  student1 = Student("John Doe")
30  student1.setMarks("Math", 85)
31  student1.setMarks("Science", 90)
32  student1.setMarks("English", 88)
33
34  print(student1.getMarks("Math"))
35  print(student1.getMarks("History"))
36  print(student1.avgMarks())
37  student1.showResults()
```

SAMPLE OUTPUT

Terminal Output

```
85
No marks available for this subject.
87.66666666666667
Results for John Doe:
Math:  85
Science:  90
English:  88
Average Marks:  87.66666666666667
```

RESULT

The Student class implementation with method encapsulation was verified successfully.

EXPERIMENT NO: 13 **Date:** _____

INHERITANCE

AIM

Write a program to create a vehicle class with variables name, speed, mileage and methods setCapacity() and getFare(), create a class bus that inherits from the Vehicle class.

OBJECTIVES

- To implement OOP inheritance between superclasses and subclasses.
- To leverage superclass methods using super().
- To demonstrate method overriding and specialized behavior in derived classes.

ALGORITHM

Step 1 :- Initialize inheritance program to model base Vehicle class and derived Bus child class behaviors.

Step 2 :- Define base Vehicle class with constructor setting name, speed, mileage, and initial capacity.

Step 3 :- Add setCapacity method to store passenger limit and getFare method calculating base fare.

Step 4 :- Create child Bus class inheriting from Vehicle, overriding getFare method with additional charge.

Step 5 :- Create Vehicle object vehicle1, set capacity to five, and display vehicle properties and fare.

Step 6 :- Create Bus object bus1, set capacity to fifty, and display inherited properties and calculated fare.

Step 7 :- Conclude object oriented inheritance program execution and release class instances back to memory heap.

PROGRAM

```
class Vehicle:
    def __init__(self, name, speed, mileage):
        self.name = name
        self.speed = speed
        self.mileage = mileage
        self.capacity = 0

    def setCapacity(self, capacity):
        self.capacity = capacity

    def getFare(self):
        return self.capacity * 100

class Bus(Vehicle):
    def getFare(self):
        base_fare = super().getFare()
        return base_fare * 1.1
```

```
18
19 vehicle1 = Vehicle("Car", 150, 15)
20 vehicle1.setCapacity(5)
21 print(f"Vehicle Name: {vehicle1.name}")
22 print(f"Speed: {vehicle1.speed} km/h")
23 print(f"Mileage: {vehicle1.mileage} km/l")
24 print(f"Capacity: {vehicle1.capacity}")
25 print(f"Fare: {vehicle1.getFare()}")
26
27 bus1 = Bus("School Bus", 80, 10)
28 bus1.setCapacity(50)
29 print(f"\nBus Name: {bus1.name}")
30 print(f"Speed: {bus1.speed} km/h")
31 print(f"Mileage: {bus1.mileage} km/l")
32 print(f"Capacity: {bus1.capacity}")
33 print(f"Fare: {bus1.getFare()}")
```

SAMPLE OUTPUT

Terminal Output

```
Vehicle Name:  Car
Speed:  150 km/h
Mileage:  15 km/l
Capacity:  5
Fare:  500

Bus Name:  School Bus
Speed:  80 km/h
Mileage:  10 km/l
Capacity:  50
Fare:  5500.0
```

RESULT

The OOP inheritance hierarchy between Vehicle and Bus classes was successfully demonstrated.

EXPERIMENT NO: 14 **Date:** _____

NUMPY

AIM

Write a program to create a matrix using numpy and do the following operations: Perform matrix addition, subtraction and multiplication, Find transpose of matrix, Calculates the inverse of matrix, Calculates the determinant of matrix, Transforms a matrix into 1d array (flatten)

OBJECTIVES

- To utilize NumPy library for numerical array processing.
- To perform fundamental matrix operations (addition, subtraction, dot product).
- To compute linear algebra functions (transpose, inverse, determinant, flattening).

ALGORITHM

Step 1 :- Initialize matrix computation module using NumPy library for matrix arithmetic and linear algebra operations.

Step 2 :- Import numpy module under standard alias np to access array creation and matrix functions.

Step 3 :- Create two 2D arrays matrix1 and matrix2 with 2x2 integer values for calculations.

Step 4 :- Perform element wise addition, subtraction, and dot product multiplication using NumPy built in functions.

Step 5 :- Compute matrix transpose using np.transpose function to swap rows and columns of matrix1.

Step 6 :- Compute matrix inverse using np.linalg.inv and matrix determinant using np.linalg.det on matrix1.

Step 7 :- Transform 2D array matrix1 into 1D array flattened using matrix flatten method call.

Step 8 :- Print original matrices and calculated addition, subtraction, multiplication, transpose, inverse, determinant, and flattened outputs.

Step 9 :- Conclude NumPy matrix operations program execution and free array memory allocations from environment heap.

PROGRAM

```
import numpy as np

# Create matrices
matrix1 = np.array([[1, 2], [3, 4]])
matrix2 = np.array([[5, 6], [7, 8]])

# Perform matrix addition
addition = np.add(matrix1, matrix2)

```

```
# Perform matrix subtraction
subtraction = np.subtract(matrix1, matrix2)

# Perform matrix multiplication
multiplication = np.dot(matrix1, matrix2)

# Find transpose of matrix1
transpose = np.transpose(matrix1)

# Calculate the inverse of matrix1
inverse = np.linalg.inv(matrix1)

# Calculate the determinant of matrix1
determinant = np.linalg.det(matrix1)

# Transform matrix1 into a 1D array (flatten)
flattened = matrix1.flatten()

# Display results
print("Matrix 1:\n", matrix1)
print("\nMatrix 2:\n", matrix2)
print("\nMatrix Addition:\n", addition)
print("\nMatrix Subtraction:\n", subtraction)
print("\nMatrix Multiplication:\n", multiplication)
print("\nTranspose of Matrix 1:\n", transpose)
print("\nInverse of Matrix 1:\n", inverse)
print("\nDeterminant of Matrix 1:\n", determinant)
print("\nFlattened Matrix 1:\n", flattened)
```

SAMPLE OUTPUT

Terminal Output

```
Matrix 1:
[1 2

4
]


Matrix 2:
[5 6

8

]


Matrix Addition:
[ 6 8

12

]


Matrix Subtraction:
[-4 -4

-4
]


Matrix Multiplication:
[19 22

50









]


Transpose of Matrix 1:
[1 3

4
]


Inverse of Matrix 1:
[-2.  1.

-0.5
]


Determinant of Matrix 1:
-2.0


Flattened Matrix 1:
2 3 4
```

RESULT

The matrix operations program using NumPy library was executed and results verified.

EXPERIMENT NO: 15 **Date:** _____

PANDA MODULE

AIM

Write a python program to read a .csv file using panda module and print the first and last five lines of a file

OBJECTIVES

- To leverage Pandas library for structured data manipulation.
- To load CSV datasets into DataFrames using $read_csv$.
- To inspect head and tail slices of dataset files.

ALGORITHM

Step 1 :- Initialize data processing program using Pandas module to read CSV file records and preview data lines.

Step 2 :- Import pandas library under alias pd to utilize DataFrame loading and inspection utility functions.

Step 3 :- Read external CSV dataset file using pd.$read_csv function and store data in DataFrame df$.

Step 4 :- Call head method on DataFrame df to retrieve the first five rows of CSV dataset.

Step 5 :- Call tail method on DataFrame df to retrieve the last five rows of CSV dataset.

Step 6 :- Print first five lines and last five lines of dataset to standard terminal console display.

Step 7 :- Terminate execution of Pandas file processing script and clear DataFrame structures from system memory.

PROGRAM

```
import pandas as pd

filename = 'example.csv'
df = pd.read_csv(filename)

print("First five lines of the file:")
print(df.head())

print("\nLast five lines of the file:")
print(df.tail())
```

SAMPLE OUTPUT

Terminal Output

```
First five lines of the file:
Name Age City
0 Alice 30 New York
1 Bob 25 Los Angeles
2 Charlie 35 Chicago
3 David 40 Houston
4 Eve 28 Phoenix

Last five lines of the file:
Name Age City
5 Frank 33 Philadelphia
6 Grace 45 San Antonio
7 Hannah 27 San Diego
8 Ivan 50 Dallas
9 Judy 29 San Jose
```

RESULT

The Pandas CSV data reading and head/tail preview program was verified successfully.

EXPERIMENT NO: 16 **Date:** _____

MATPLOTLIB

AIM

Write a program to draw geometric shapes using matplotlib

OBJECTIVES

- To initialize Matplotlib figure and subplot axis objects.
- To render geometric patch primitives (Rectangle, Circle, Polygon).
- To format plot aesthetics including labels, title, grid, aspect ratio, and show display.

ALGORITHM

Step 1 :- Initialize graphic visualization program using Matplotlib module to render geometric shapes on plot canvas.

Step 2 :- Import matplotlib.pyplot module under alias plt and numpy module under alias np for drawing.

Step 3 :- Create figure and subplots axis objects using plt.subplots function to define plot drawing canvas.

Step 4 :- Create Rectangle patch specifying position, width, height, color, and alpha transparency, adding to axis.

Step 5 :- Create Circle patch specifying center position, radius, color, and transparency, adding to plot axis.

Step 6 :- Create Polygon patch with NumPy array triangle coordinates, adding red triangle patch to axis.

Step 7 :- Set x and y axis limits from zero to one and set equal aspect ratio scaling.

Step 8 :- Set grid, axis labels, title, and display rendered figure window using plt.show function.

Step 9 :- Close graphic visualization window and terminate program execution releasing plot resources back to system.

PROGRAM

```
import matplotlib.pyplot as plt
import numpy as np

fig, ax = plt.subplots()

rectangle = plt.Rectangle((0.2, 0.5), 0.4, 0.3, color='blue', alpha=0.3)
ax.add_patch(rectangle)

circle = plt.Circle((0.8, 0.2), 0.1, color='green', alpha=0.5)
ax.add_patch(circle)

```

```
triangle_points = np.array([[0.6, 0.8], [0.8, 0.8], [0.7, 0.9], [0.6, 0.8]])
triangle = plt.Polygon(triangle_points, closed=True, color='red', alpha=0.4)
ax.add_patch(triangle)

ax.set_xlim(0, 1)
ax.set_ylim(0, 1)
ax.set_aspect('equal', adjustable='box')

ax.grid(True)
ax.set_xlabel('X-axis')
ax.set_ylabel('Y-axis')
ax.set_title('Geometric Shapes')

plt.show()
```

SAMPLE OUTPUT

Terminal Output

```
[Plot Window Displayed:  Rendered Blue Rectangle, Green Circle, and Red Triangle
shapes on grid axes]
```

RESULT

The Matplotlib geometric shape rendering program was successfully executed and visualized.